

Mini libreria per sistemi lineari

Solutori iterativi per matrici simmetriche e definite positive

Metodi del Calcolo Scientifico — AA 2025–2026

Progetto 1 (alternativo)

Sommario

Implementazione e validazione di quattro solutori iterativi — Jacobi, Gauss–Seidel, Gradiente e Gradiente coniugato — per sistemi lineari $A\mathbf{x} = \mathbf{b}$ con A simmetrica e definita positiva (SPD). La libreria è scritta in **Python**: per le sole strutture dati (matrice sparsa CSR, vettori) e le operazioni elementari ($A\mathbf{v}$, prodotto scalare, norma) ci si appoggia a `scipy.sparse/numpy`, mentre gli algoritmi risolutivi sono interamente sviluppati nel progetto. La validazione segue la procedura standard (soluzione esatta nota $\mathbf{x} = [1, \dots, 1]$, $\mathbf{b} = A\mathbf{x}$, $\mathbf{x}^{(0)} = \mathbf{0}$) sulle quattro matrici fornite e sulle tolleranze $\{10^{-4}, 10^{-6}, 10^{-8}, 10^{-10}\}$.

Indice

1	Struttura della libreria	1
1.1	Architettura	1
1.2	Note implementative rilevanti	2
2	Richiamo teorico dei quattro metodi	2
2.1	Metodi stazionari: lo splitting $A = P - N$	2
2.2	Metodi del gradiente: risolvere = minimizzare	3
3	Metodologia di validazione	3
3.1	Le matrici di test	3
4	Risultati	4
4.1	Grafici di sintesi	7
5	Commenti	11
6	Domande d’esame anticipate (Q&A)	12
6.1	Vincoli della consegna	12
6.2	Architettura e implementazione	12
6.3	Teoria dei metodi	13
6.4	Test “on-the-fly”	13
7	Come eseguire	13
8	Conclusioni	14

1 Struttura della libreria

Il progetto è scritto in **Python**. Per le sole **strutture dati** (matrice sparsa CSR, vettori) e le **operazioni elementari** (prodotto matrice-vettore Av , prodotto scalare, norma) ci si appoggia a `scipy.sparse/numpy`, come consentito esplicitamente dalla consegna. **Gli algoritmi risolutivi sono interamente implementati nel progetto** (file `solvers.py`): non viene usato alcun solutore di sistemi lineari di `scipy/numpy` (niente `spsolve`, `cg`, `numpy.linalg.solve`, ecc.).

```
assignment1MCS/
  solvers.py          # i 4 metodi + criterio di arresto + sweep GS (
    numba)
  run_assignment.py   # eseguibile: matrici + tolleranze -> risultati
    + CSV
  plot_results.py     # generazione grafici
  test_assignment.py  # test automatici
  analyze_cond.py     # stima di cond(A) (solo per i commenti)
  dati/              # spa1, spa2, vem1, vem2 (.mtx)
  results/           # results.csv + plots/*.png
  Teoria.md          # documento teorico divulgativo
  Relazione.tex      # questo documento
  README.md
```

1.1 Architettura

La libreria **non è una sequenza di funzioni indipendenti**, ma una piccola gerarchia di classi coesa, costruita su un'osservazione: tutti i metodi condividono lo stesso scheletro e differiscono *solo* nel passo di aggiornamento $\mathbf{x}^{(k)} \rightarrow \mathbf{x}^{(k+1)}$.

- **IterativeSolver** (classe base astratta) implementa **una sola volta** lo scheletro comune nel metodo `solve()`: vettore iniziale nullo $\mathbf{x}^{(0)} = \mathbf{0}$; ciclo che si arresta quando il residuo scalato $\|\mathbf{b} - A\mathbf{x}^{(k)}\|/\|\mathbf{b}\| < \text{tol}$, oppure quando si superano `max_iter` iterazioni (segnalando la non convergenza); validazione, misura del tempo, costruzione del risultato.

```
x^(0) = 0
finche' ||b - A x^(k)|| / ||b|| >= tol e k < max_iter:
    x^(k+1) = aggiornamento(x^(k))
    k += 1
```

- **_Iteration** (interfaccia astratta) incapsula lo stato e il *passo* di un metodo (pattern *template method*). Le quattro classi concrete — **Jacobi**, **GaussSeidel**, **Gradient**, **ConjugateGradient** — forniscono soltanto la propria regola di aggiornamento. Aggiungere un quinto metodo significa scrivere una sola classe, senza toccare lo scheletro.
- **SolverResult** (dataclass) raccoglie in modo uniforme l'esito: soluzione, iterazioni, tempo, residuo relativo finale, convergenza.

Le istanze dei quattro metodi sono raccolte nella tupla **SOLVERS**, su cui iterano sia il runner sia i test. Le parti di servizio comuni sono fattorizzate (`_validate_for_iteration`: matrice quadrata e diagonale non nulla; `_safe_norm`: norma di **b**, calcolata una sola volta).

1.2 Note implementative rilevanti

- **Sparsità preservata**. Nessun metodo fattorizza o modifica **A**: la matrice è usata solo per prodotti Av , il cui costo è proporzionale al numero di non-zeri e non a n^2 (niente *fill-in*).

- **Un solo matvec per iterazione.** Per Jacobi, Gradiente e Gradiente coniugato il residuo successivo si ottiene con la ricorrenza $\mathbf{r}^{(k+1)} = \mathbf{r}^{(k)} - \alpha \mathbf{A}\mathbf{v}$, evitando un prodotto matrice–vettore in più.
- **Gauss–Seidel senza inversa, ma veloce.** Il passo $P^{-1}\mathbf{r}$ (con $P = D + L$ triangolare inferiore) è realizzato con una *sweep* in place che aggiorna $\mathbf{x}^{(k+1)}$ riusando i valori appena calcolati — equivalente a una sostituzione in avanti — **senza mai costruire** P^{-1} . Essendo intrinsecamente sequenziale (x_i dipende da $x_0, \dots, x_{i-1}$), è compilata con **numba** (`_gauss_seidel_sweep`), restando codice nostro: così non è il collo di bottiglia che sarebbe un ciclo Python puro.
- **Tempi equi.** Prima delle misure si esegue un *warm-up* che forza la compilazione JIT, così il tempo di Gauss–Seidel non include il costo (una tantum) di compilazione.

2 Richiamo teorico dei quattro metodi

Tutti i metodi assumono A SPD ($A = A^T$ e $\mathbf{v}^T \mathbf{A} \mathbf{v} > 0$ per ogni $\mathbf{v} \neq \mathbf{0}$). Il residuo è $\mathbf{r}^{(k)} = \mathbf{b} - \mathbf{A}\mathbf{x}^{(k)}$ e il criterio di arresto è sul residuo scalato $\|\mathbf{r}^{(k)}\|/\|\mathbf{b}\| < \text{tol}$.

2.1 Metodi stazionari: lo splitting $A = P - N$

Si spezza $A = P - N$ con P facile da invertire; il sistema diventa l'iterazione

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + P^{-1}\mathbf{r}^{(k)}.$$

Il metodo converge per ogni $\mathbf{x}^{(0)}$ se e solo se il raggio spettrale della *matrice di iterazione* $B = P^{-1}N$ soddisfa $\rho(B) < 1$.

- **Jacobi:** $P = D$ (diagonale). D^{-1} è banale (reciproco degli elementi diagonali), il passo è completamente vettoriale e parallelizzabile. La sola SPD *non* garantisce $\rho(B) < 1$ (condizione sufficiente: anche $2D - A$ SPD, oppure dominanza diagonale).
- **Gauss–Seidel:** $P = D + L$ (triangolare inferiore). Usa subito le componenti appena aggiornate: tipicamente *circa metà* delle iterazioni di Jacobi. Se A è SPD, Gauss–Seidel converge **sempre**. Il passo è una sostituzione in avanti, intrinsecamente sequenziale.

2.2 Metodi del gradiente: risolvere = minimizzare

Per A SPD, risolvere $\mathbf{A}\mathbf{x} = \mathbf{b}$ equivale a minimizzare

$$\varphi(\mathbf{y}) = \frac{1}{2} \mathbf{y}^T \mathbf{A} \mathbf{y} - \mathbf{b}^T \mathbf{y}, \quad \nabla \varphi(\mathbf{y}) = \mathbf{A} \mathbf{y} - \mathbf{b},$$

che si annulla in $\mathbf{A}\mathbf{y} = \mathbf{b}$ ed è un minimo (paraboloide a “ciotola”) proprio perché A è definita positiva.

- **Gradiente** (*steepest descent*): direzione = $-\nabla \varphi = \mathbf{r}^{(k)}$, passo ottimo lungo la retta

$$\alpha_k = \frac{\mathbf{r}^{(k)} \cdot \mathbf{r}^{(k)}}{\mathbf{r}^{(k)} \cdot \mathbf{A} \mathbf{r}^{(k)}}, \quad \mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha_k \mathbf{r}^{(k)}.$$

Converge sempre se A è SPD, ma su matrici mal condizionate la direzione di massima discesa non punta al fondo della valle: si genera il caratteristico *zig-zag* e la velocità dipende dal numero di condizionamento $\text{cond}(A) = \lambda_{\max}/\lambda_{\min}$.

- **Gradiente coniugato** (CG): direzioni A -coniugate ($\mathbf{d}_i^T \mathbf{A} \mathbf{d}_j = 0$ per $i \neq j$): una volta ottimizzata una direzione non la si “rovina” più, niente zig-zag. Per A SPD di dimensione n , CG raggiunge la soluzione esatta in **al più n iterazioni**; in pratica ne bastano molte meno e il numero di iterazioni scala come $\sqrt{\text{cond}(A)}$.

3 Metodologia di validazione

Procedura standard richiesta dalla consegna (step 1–4), per ogni matrice:

1. soluzione esatta $\mathbf{x} = [1, 1, \dots, 1]$;
 2. termine noto $\mathbf{b} = A\mathbf{x}$;
 3. risoluzione con i quattro metodi a partire da $\mathbf{x}^{(0)} = \mathbf{0}$;
 4. misura di **errore relativo** $\|\mathbf{x}_{\text{calc}} - \mathbf{x}\|/\|\mathbf{x}\|$, **iterazioni** e **tempo**.
- Tolleranze: `tol` $\in \{10^{-4}, 10^{-6}, 10^{-8}, 10^{-10}\}$.
 - `max_iter` = 20000.
 - Criterio di arresto: residuo scalato $\|\mathbf{b} - A\mathbf{x}^{(k)}\|/\|\mathbf{b}\| < \text{tol}$.

Ambiente di calcolo (lo stesso per tutti i test): Intel Core i7-1165G7 @ 2.80 GHz, 16 GB RAM, Windows 11, Python 3.12, numpy, scipy, numba.

3.1 Le matrici di test

Tabella 1: Caratteristiche delle matrici di test (autovalori e $\text{cond}(A)$ stimati con `analyze_cond.py`).

Matrice	n	non-zero	$\lambda_{\min}$	$\lambda_{\max}$	$\text{cond}(A)$
spa1	1000	182 434	$4.88 \cdot 10^{-1}$	$9.99 \cdot 10^2$	$2.05 \cdot 10^3$
spa2	3000	1 633 298	$2.12 \cdot 10^0$	$3.00 \cdot 10^3$	$1.41 \cdot 10^3$
vem1	1681	13 385	$1.23 \cdot 10^{-2}$	$4.00 \cdot 10^0$	$3.25 \cdot 10^2$
vem2	2601	21 225	$7.89 \cdot 10^{-3}$	$4.00 \cdot 10^0$	$5.07 \cdot 10^2$

Le **spa*** sono molto più dense (≈ 180 e ≈ 540 non-zero per riga) e con condizionamento maggiore; le **vem*** sono molto più sparse (≈ 8 non-zero per riga) e meglio condizionate.

4 Risultati

Tabelle complete estratte da `results/results.csv`. Tutti i metodi sono arrivati a convergenza (`conv` = sì) entro `max_iter` = 20000. I tempi sono in secondi sull’ambiente indicato (soggetti a variazioni di runtime fra esecuzioni).

Tabella 2: **spa1** — $n = 1000$, $\text{nnz} = 182\,434$.

tol	metodo	iter	err. rel.	res. scal.	t [s]
10^{-4}	Jacobi	115	$1.77 \cdot 10^{-3}$	$9.48 \cdot 10^{-5}$	0.080
10^{-4}	Gauss–Seidel	9	$1.82 \cdot 10^{-2}$	$7.79 \cdot 10^{-5}$	0.017
10^{-4}	Gradiente	143	$3.46 \cdot 10^{-2}$	$9.91 \cdot 10^{-5}$	0.093
10^{-4}	Gradiente coniugato	49	$2.08 \cdot 10^{-2}$	$9.78 \cdot 10^{-5}$	0.031
10^{-6}	Jacobi	181	$1.80 \cdot 10^{-5}$	$9.62 \cdot 10^{-7}$	0.110
10^{-6}	Gauss–Seidel	17	$1.30 \cdot 10^{-4}$	$5.58 \cdot 10^{-7}$	0.031
10^{-6}	Gradiente	3577	$9.68 \cdot 10^{-4}$	$9.98 \cdot 10^{-7}$	2.172
10^{-6}	Gradiente coniugato	134	$2.55 \cdot 10^{-5}$	$9.74 \cdot 10^{-7}$	0.091
10^{-8}	Jacobi	247	$1.82 \cdot 10^{-7}$	$9.76 \cdot 10^{-9}$	0.168
10^{-8}	Gauss–Seidel	24	$1.71 \cdot 10^{-6}$	$7.34 \cdot 10^{-9}$	0.038
10^{-8}	Gradiente	8233	$9.82 \cdot 10^{-6}$	$9.99 \cdot 10^{-9}$	5.819
10^{-8}	Gradiente coniugato	177	$1.32 \cdot 10^{-7}$	$9.03 \cdot 10^{-9}$	0.140
10^{-10}	Jacobi	313	$1.85 \cdot 10^{-9}$	$9.91 \cdot 10^{-11}$	0.241
10^{-10}	Gauss–Seidel	31	$2.25 \cdot 10^{-8}$	$9.65 \cdot 10^{-11}$	0.058
10^{-10}	Gradiente	12919	$9.82 \cdot 10^{-8}$	$9.99 \cdot 10^{-11}$	10.171
10^{-10}	Gradiente coniugato	200	$1.20 \cdot 10^{-9}$	$7.94 \cdot 10^{-11}$	0.156

Tabella 3: **spa2** — $n = 3000$, $\text{nnz} = 1\,633\,298$.

tol	metodo	iter	err. rel.	res. scal.	t [s]
10^{-4}	Jacobi	36	$1.77 \cdot 10^{-3}$	$9.54 \cdot 10^{-5}$	0.259
10^{-4}	Gauss–Seidel	5	$2.60 \cdot 10^{-3}$	$4.14 \cdot 10^{-5}$	0.088
10^{-4}	Gradiente	161	$1.81 \cdot 10^{-2}$	$9.90 \cdot 10^{-5}$	1.176
10^{-4}	Gradiente coniugato	42	$9.82 \cdot 10^{-3}$	$9.91 \cdot 10^{-5}$	0.306
10^{-6}	Jacobi	57	$1.67 \cdot 10^{-5}$	$9.00 \cdot 10^{-7}$	0.419
10^{-6}	Gauss–Seidel	8	$5.14 \cdot 10^{-5}$	$7.85 \cdot 10^{-7}$	0.146
10^{-6}	Gradiente	1949	$6.69 \cdot 10^{-4}$	$9.99 \cdot 10^{-7}$	12.249
10^{-6}	Gradiente coniugato	122	$1.20 \cdot 10^{-4}$	$9.09 \cdot 10^{-7}$	0.692
10^{-8}	Jacobi	78	$1.57 \cdot 10^{-7}$	$8.49 \cdot 10^{-9}$	0.480
10^{-8}	Gauss–Seidel	12	$2.79 \cdot 10^{-7}$	$4.27 \cdot 10^{-9}$	0.178
10^{-8}	Gradiente	5087	$6.87 \cdot 10^{-6}$	$9.98 \cdot 10^{-9}$	34.083
10^{-8}	Gradiente coniugato	196	$5.59 \cdot 10^{-7}$	$9.60 \cdot 10^{-9}$	1.387
10^{-10}	Jacobi	99	$1.48 \cdot 10^{-9}$	$8.01 \cdot 10^{-11}$	0.706
10^{-10}	Gauss–Seidel	15	$5.57 \cdot 10^{-9}$	$8.52 \cdot 10^{-11}$	0.262
10^{-10}	Gradiente	8285	$6.94 \cdot 10^{-8}$	$9.99 \cdot 10^{-11}$	58.376
10^{-10}	Gradiente coniugato	240	$5.32 \cdot 10^{-9}$	$9.82 \cdot 10^{-11}$	1.613

Tabella 4: **vem1** — $n = 1681$, $\text{nnz} = 13\,385$.

tol	metodo	iter	err. rel.	res. scal.	t [s]
10^{-4}	Jacobi	1314	$3.54 \cdot 10^{-3}$	$9.99 \cdot 10^{-5}$	0.084
10^{-4}	Gauss–Seidel	659	$3.51 \cdot 10^{-3}$	$9.93 \cdot 10^{-5}$	0.136
10^{-4}	Gradiente	890	$2.70 \cdot 10^{-3}$	$9.93 \cdot 10^{-5}$	0.098
10^{-4}	Gradiente coniugato	38	$4.08 \cdot 10^{-5}$	$7.11 \cdot 10^{-5}$	0.003
10^{-6}	Jacobi	2433	$3.54 \cdot 10^{-5}$	$9.99 \cdot 10^{-7}$	0.219
10^{-6}	Gauss–Seidel	1218	$3.53 \cdot 10^{-5}$	$9.99 \cdot 10^{-7}$	0.197
10^{-6}	Gradiente	1612	$2.71 \cdot 10^{-5}$	$9.97 \cdot 10^{-7}$	0.155
10^{-6}	Gradiente coniugato	45	$3.73 \cdot 10^{-7}$	$8.90 \cdot 10^{-7}$	0.004
10^{-8}	Jacobi	3552	$3.54 \cdot 10^{-7}$	$9.99 \cdot 10^{-9}$	0.273
10^{-8}	Gauss–Seidel	1778	$3.52 \cdot 10^{-7}$	$9.96 \cdot 10^{-9}$	0.328
10^{-8}	Gradiente	2336	$2.70 \cdot 10^{-7}$	$9.90 \cdot 10^{-9}$	0.231
10^{-8}	Gradiente coniugato	53	$2.83 \cdot 10^{-9}$	$7.80 \cdot 10^{-9}$	0.005
10^{-10}	Jacobi	4671	$3.54 \cdot 10^{-9}$	$9.99 \cdot 10^{-11}$	0.352
10^{-10}	Gauss–Seidel	2338	$3.51 \cdot 10^{-9}$	$9.94 \cdot 10^{-11}$	0.370
10^{-10}	Gradiente	3058	$2.71 \cdot 10^{-9}$	$9.97 \cdot 10^{-11}$	0.297
10^{-10}	Gradiente coniugato	59	$2.19 \cdot 10^{-11}$	$6.91 \cdot 10^{-11}$	0.005

Tabella 5: **vem2** — $n = 2601$, $\text{nnz} = 21\,225$.

tol	metodo	iter	err. rel.	res. scal.	t [s]
10^{-4}	Jacobi	1927	$4.97 \cdot 10^{-3}$	$9.99 \cdot 10^{-5}$	0.198
10^{-4}	Gauss–Seidel	965	$4.95 \cdot 10^{-3}$	$9.98 \cdot 10^{-5}$	0.223
10^{-4}	Gradiente	1308	$3.81 \cdot 10^{-3}$	$9.98 \cdot 10^{-5}$	0.159
10^{-4}	Gradiente coniugato	47	$5.73 \cdot 10^{-5}$	$9.19 \cdot 10^{-5}$	0.006
10^{-6}	Jacobi	3676	$4.97 \cdot 10^{-5}$	$9.99 \cdot 10^{-7}$	0.381
10^{-6}	Gauss–Seidel	1840	$4.94 \cdot 10^{-5}$	$9.96 \cdot 10^{-7}$	0.557
10^{-6}	Gradiente	2438	$3.79 \cdot 10^{-5}$	$9.92 \cdot 10^{-7}$	0.321
10^{-6}	Gradiente coniugato	56	$4.74 \cdot 10^{-7}$	$8.54 \cdot 10^{-7}$	0.009
10^{-8}	Jacobi	5425	$4.97 \cdot 10^{-7}$	$9.99 \cdot 10^{-9}$	0.738
10^{-8}	Gauss–Seidel	2714	$4.96 \cdot 10^{-7}$	$9.99 \cdot 10^{-9}$	0.720
10^{-8}	Gradiente	3566	$3.81 \cdot 10^{-7}$	$9.97 \cdot 10^{-9}$	0.588
10^{-8}	Gradiente coniugato	66	$4.30 \cdot 10^{-9}$	$8.60 \cdot 10^{-9}$	0.011
10^{-10}	Jacobi	7174	$4.96 \cdot 10^{-9}$	$9.98 \cdot 10^{-11}$	0.819
10^{-10}	Gauss–Seidel	3589	$4.95 \cdot 10^{-9}$	$9.97 \cdot 10^{-11}$	0.230
10^{-10}	Gradiente	4696	$3.80 \cdot 10^{-9}$	$9.94 \cdot 10^{-11}$	0.157
10^{-10}	Gradiente coniugato	74	$2.25 \cdot 10^{-11}$	$5.90 \cdot 10^{-11}$	0.002

4.1 Grafici di sintesi

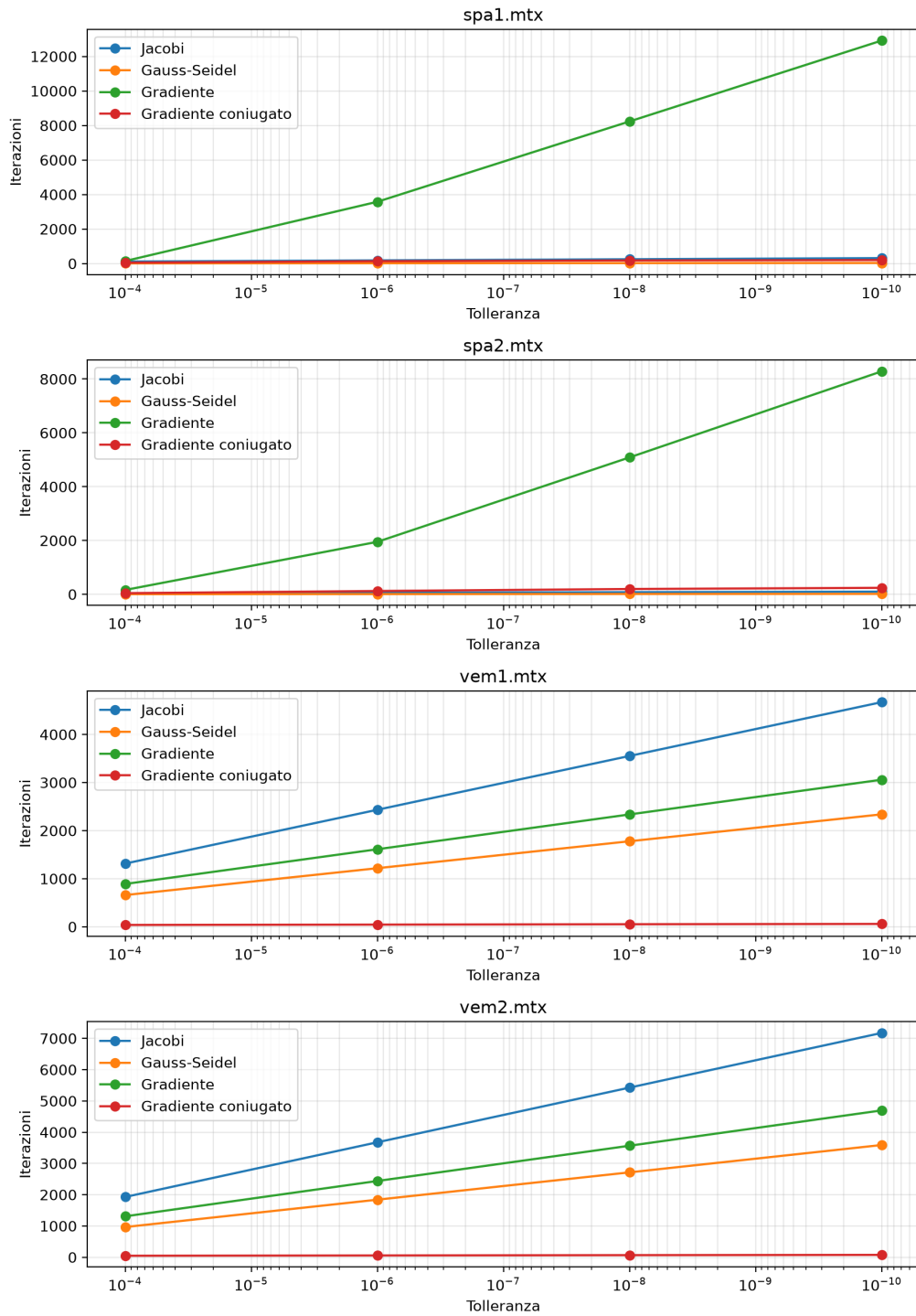


Figura 1: Numero di iterazioni al variare di `tol` (asse y in scala logaritmica).

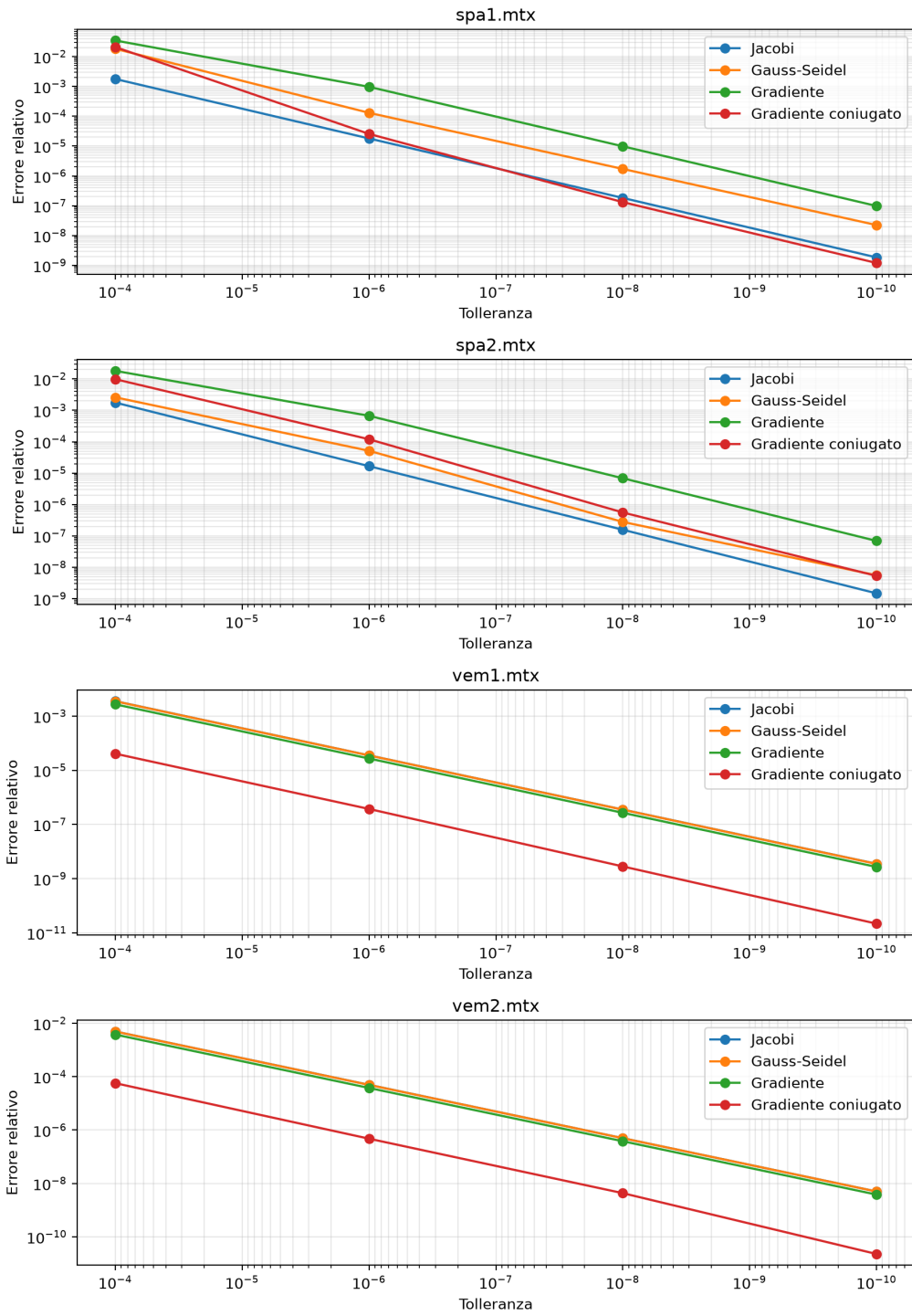


Figura 2: Errore relativo $\|\mathbf{x}_{\text{calc}} - \mathbf{x}\|/\|\mathbf{x}\|$ al variare di tol (scala logaritmica).

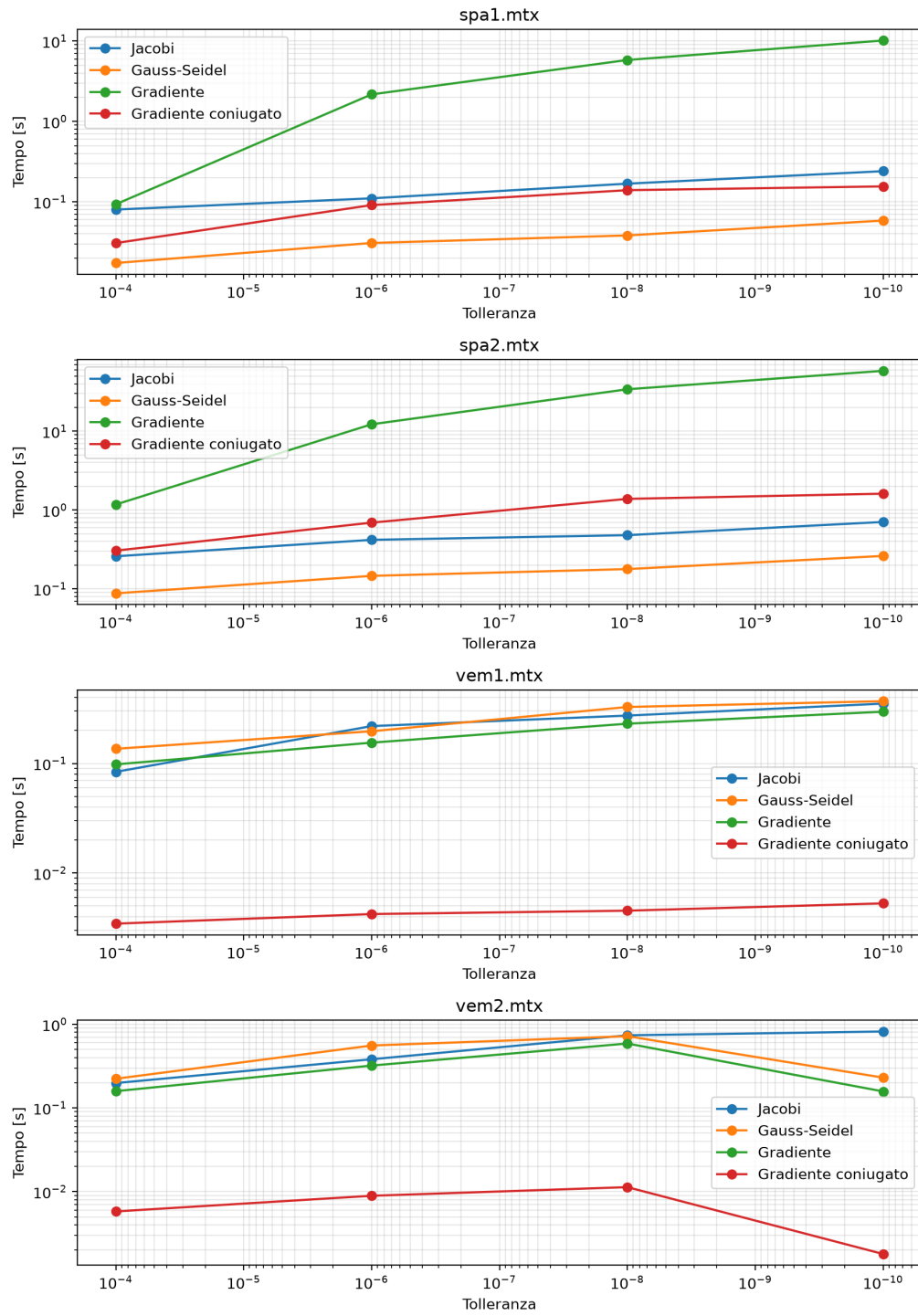


Figura 3: Tempo di calcolo al variare di tol (scala logaritmica).

5 Commenti

Tutti e quattro i metodi convergono su tutte le matrici e tutte le tolleranze, entro `max_iter = 20000`. Le matrici di test sono quindi “trattabili” anche dai metodi più semplici. Emergono comunque comportamenti molto diversi.

1) Gauss–Seidel $\approx$ metà delle iterazioni di Jacobi. È quanto previsto dalla teoria: riusingo immediatamente le componenti appena calcolate fa più “lavoro utile” per iterazione. Es. `vem2 @10-8`: Jacobi 5425 vs Gauss–Seidel 2714 iterazioni (rapporto ≈ 2.0). Il singolo passo di Gauss–Seidel è un po’ più costoso (sweep sequenziale), perciò sulle matrici molto sparse `vem` i tempi totali dei due metodi si equivalgono, mentre sulle `spa` (dove GS converge in pochissime iterazioni) GS è nettamente il più rapido.

2) Due “regimi” opposti tra le famiglie di matrici. I metodi stazionari (Jacobi/Gauss–Seidel) e quelli del gradiente rispondono a proprietà diverse della matrice:

- Su `spa1/spa2` Jacobi e Gauss–Seidel sono fulminei (5–313 iterazioni), mentre il Gradiente è lentissimo (fino a ~ 12900 iterazioni). Le `spa` sono fortemente *dominanti diagonali* $\Rightarrow$ raggio spettrale della matrice di iterazione piccolo $\Rightarrow$ metodi stazionari rapidi; ma hanno $\text{cond}(A) \approx 2 \cdot 10^3$, e la velocità del Gradiente dipende proprio da $\text{cond}(A)$, da cui la lentezza.
- Su `vem1/vem2` la situazione si riequilibra: meno dominanza diagonale rende Jacobi/Gauss–Seidel più lenti (migliaia di iterazioni), mentre il minore $\text{cond}(A) \approx 3\text{--}5 \cdot 10^2$ rende il Gradiente competitivo.

Questo conferma che la velocità di Jacobi/Gauss–Seidel è governata dalla **dominanza diagonale** ($\rho(P^{-1}N)$), mentre quella di Gradiente/Gradiente coniugato è governata dal **numero di condizionamento**.

3) Il Gradiente coniugato è il metodo più robusto e regolare. Converge sempre in **poches decine di iterazioni** (38–240) indipendentemente dalla matrice, con crescita lentissima al ridursi di `tol`. Il numero di iterazioni scala come $\sqrt{\text{cond}(A)}$ (coerente: `spa1`, $\text{cond} \approx 2 \cdot 10^3$, richiede più passi di `vem1`, $\text{cond} \approx 3 \cdot 10^2$) ed è sempre ben al di sotto del limite teorico di n iterazioni. È il vincitore netto sulle matrici mal condizionate `vem`.

4) Residuo piccolo $\neq$ errore piccolo. A parità di `tol`, l’errore relativo sulla soluzione **non** è uguale fra i metodi e, per i metodi stazionari, vale circa `tol` $\times$ cost, con cost legato al condizionamento (relazione errore $\lesssim \text{cond}(A) \cdot \text{residuo}$). Esempi a `tol = 10-6`: su `vem2` l’errore è $\sim 5 \cdot 10^{-5}$ (circa $50\times$ il residuo); su `spa1` il Gradiente ha errore $\sim 10^{-3}$ (circa $1000\times$ il residuo). Il Gradiente coniugato raggiunge invece errori molto più piccoli (fino a $\sim 10^{-11}$), perché tende a ridurre simultaneamente tutte le componenti spettrali dell’errore.

5) Tempi di calcolo. In secondi assoluti il quadro dipende dalla densità:

- sulle `spa` (dense) il prodotto matrice–vettore è caro, quindi il Gradiente, che fa molte iterazioni, arriva a oltre 50 s (`spa2 @10-10`), mentre Gauss–Seidel resta sotto 0.3 s e CG sotto 1.7 s;
- sulle `vem` (molto sparse) tutti i metodi sono rapidi (< 1 s); CG è comunque il più veloce in assoluto (≤ 0.011 s) grazie al bassissimo numero di iterazioni.

In sintesi, su queste matrici il **Gradiente coniugato è la scelta migliore in ogni scenario**; tra i metodi stazionari, Gauss–Seidel domina Jacobi; il Gradiente semplice è penalizzato quando la matrice è mal condizionata (effetto “zig-zag”).

6 Domande d'esame anticipate (Q&A)

Raccolta delle domande che il docente può porre all'orale, con risposta sintetica e puntatori al codice.

6.1 Vincoli della consegna

Q — Python è ammesso?

Sì: la consegna elenca esplicitamente «C++, Fortran, Java, **Python**, etc.».

Q — Avete usato un solutore già pronto (vietato dalla consegna)?

No. `scipy/numpy` sono usati **solo** per le strutture dati (lettura `.mtx`, matrice sparsa CSR) e le operazioni elementari (prodotto matrice-vettore Ax , prodotto scalare, norma). I quattro algoritmi sono scritti interamente in `solvers.py`. Non si usano `scipy.sparse.linalg.cg`, `spsolve`, `numpy.linalg.solve` o simili. È esattamente il caso «la libreria di base fornisce *solamente* la struttura dati» richiesto al punto (a).

Q — Perché Ax è lecito e un solver no?

Ax è un'operazione elementare fra matrice e vettore, non la *risoluzione* di un sistema. La consegna vieta i metodi che risolvono $Ax = b$, non l'aritmetica di base.

Q — Il vettore iniziale è davvero nullo? E il criterio di arresto?

Sì: $x^{(0)} = 0$ (`solvers.py`, riga 97). Il criterio è il residuo scalato $\|b - Ax^{(k)}\|/\|b\| < tol$ (riga 104), più il controllo $k < max_iter$ con `max_iter` = 20000 e flag `converged` = False se lo si supera — esattamente i punti (b) della consegna.

Q — Perché $x = [1, \dots, 1]$ e $b = Ax$?

È la procedura di validazione standard richiesta (step 1–4): scegliendo la soluzione esatta nota x , si può misurare l'errore vero $\|x_{calc} - x\|/\|x\|$, cosa impossibile in un problema reale dove la soluzione non si conosce. Nota: $b = A \cdot 1$ è semplicemente il vettore delle **somme di riga** di A .

6.2 Architettura e implementazione

Q — Perché non quattro funzioni separate?

Perché la consegna «valuta positivamente» un'architettura coesa. Tutti i metodi condividono lo stesso scheletro (ciclo + criterio d'arresto + misure) e differiscono solo nel passo $x^{(k)} \rightarrow x^{(k+1)}$. Lo scheletro è scritto **una volta sola** in `IterativeSolver.solve()`; ogni metodo è una classe con la sola regola di aggiornamento (pattern *template method*).

Q — Perché Gauss–Seidel è compilato con numba e gli altri no?

Il passo di Gauss–Seidel è una *sweep* sequenziale (sostituzione in avanti: x_i dipende da $x_0, \dots, x_{i-1}$ appena calcolati) e quindi **non è vettorizzabile** con `numpy`: in puro Python sarebbe un ciclo lentissimo. `numba` lo compila a codice macchina mantenendolo **codice nostro** (non è una libreria di solver). Jacobi, Gradiente e CG si esprimono già con operazioni vettoriali `numpy` efficienti.

Q — Cos'è il warm-up e perché c'è?

La prima chiamata a una funzione `numba` paga (una tantum) il costo di compilazione JIT. Il `warmup()` la forza su un problemino 3×3 **prima** delle misure di tempo, così il tempo riportato per Gauss–Seidel non include la compilazione ed è confrontabile con gli altri.

Q — Un prodotto matrice-vettore per iterazione: come?

Per Gradiente e CG il residuo successivo si ottiene per **ricorrenza** $r^{(k+1)} = r^{(k)} - \alpha(Av)$, riusando il prodotto Av già calcolato per il passo, invece di ricalcolare $b - Ax^{(k+1)}$ (che costerebbe un secondo matvec).

Q — Verificate che la matrice sia davvero SPD?

No, e va detto onestamente: `_validate_for_iteration` controlla solo che A sia **quadrata** e con **diagonale non nulla** (riga 296). La simmetria e la definita-positività sono **assunte** per ipotesi della consegna. Se servisse verificarle: la simmetria con $\|A - A^T\| \approx 0$; la definita-positività

tentando una **fattorizzazione di Cholesky** (riesce $\iff$ SPD) oppure controllando $\lambda_{\min} > 0$ (è ciò che fa `analyze_cond.py`).

6.3 Teoria dei metodi

Q — Qual è lo “splitting” $A = P - N$ di Jacobi e Gauss–Seidel?

Jacobi: $P = D$ (diagonale). Gauss–Seidel: $P = D + L$ (triangolare inferiore). In entrambi il passo è $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + P^{-1}\mathbf{r}^{(k)}$.

Q — La sola SPD non garantisce la convergenza di Jacobi: perché qui converge?

Vero: Jacobi converge $\iff \rho(D^{-1}(L + U)) < 1$, e la sola SPD non lo assicura (condizione sufficiente: anche $2D - A$ SPD, oppure dominanza diagonale). **Empiricamente** su tutte e quattro le matrici Jacobi converge $\Rightarrow$ per queste matrici quella condizione è soddisfatta. Gauss–Seidel invece, essendo A SPD, converge **sempre** per teorema.

Q — Risolvere = minimizzare: perché vale solo se A è SPD?

Perché $\varphi(\mathbf{y}) = \frac{1}{2}\mathbf{y}^T A \mathbf{y} - \mathbf{b}^T \mathbf{y}$ ha gradiente $\nabla \varphi = A \mathbf{y} - \mathbf{b}$, che si annulla in $A \mathbf{y} = \mathbf{b}$; ed è un **minimo solo se A è definita positiva**. Altrimenti φ potrebbe avere una sella o nessun minimo e i metodi del gradiente non sarebbero ben posti.

Q — Cos’è lo zig-zag del Gradiente e da cosa dipende la sua velocità?

La direzione di massima discesa punta verso la parete più vicina della valle, non verso il fondo: su valli strette (mal condizionate) si rimbalza da una parete all’altra. La velocità dipende dal **numero di condizionamento** $\text{cond}(A)$. Per questo il Gradiente è lentissimo sulle **spa** ($\text{cond} \approx 2 \cdot 10^3$): fino a $\sim 12\,900$ iter.

Q — Perché il Gradiente coniugato è così veloce? Direzioni A-coniugate?

Le direzioni sono **A-coniugate** ($\mathbf{d}_i^T A \mathbf{d}_j = 0$ per $i \neq j$): una volta ottimizzata una direzione non la si “rovina” più, niente zig-zag. Teorema: per A SPD $n \times n$, CG arriva alla soluzione esatta in **al più n iterazioni**; in pratica, grazie al raggruppamento degli autovalori, ne bastano molte meno (qui 38–240). Il numero di iterazioni scala come $\sqrt{\text{cond}(A)}$.

Q — Per CG e Gradiente il residuo del criterio d’arresto è quello “vero”?

È il residuo calcolato **per ricorrenza** ($\mathbf{r} \leftarrow \mathbf{r} - \alpha A \mathbf{v}$), non ricalcolato come $\mathbf{b} - A \mathbf{x}$ ad ogni passo (scelta per risparmiare un matvec). Su queste matrici e tolleranze la differenza con il residuo vero è trascurabile. Jacobi e Gauss–Seidel usano invece il residuo vero $\mathbf{b} - A \mathbf{x}$ ad ogni passo.

6.4 Test “on-the-fly”

Q — Sapete lanciare il codice su un’altra matrice / un’altra tolleranza all’istante?

Sì. La CLI accetta `--matrix` e `--tol` ripetibili:

```
python run_assignment.py --matrix dati/vem1.mtx --tol 1e-8
python run_assignment.py --matrix /percorso/altra.mtx --tol 1e-5 --
    tol 1e-7
```

Si può anche isolare un singolo metodo con `--method` e saltare i grafici con `--no-plots`. Funziona con qualunque `.mtx` SPD.

7 Come eseguire

Requisiti: `pip install -r requirements.txt` (numpy, scipy, numba, matplotlib).

```
# validazione completa (tutte le matrici e tolleranze) -> results/
python run_assignment.py

# una matrice / una tolleranza / un metodo specifici
python run_assignment.py --matrix dati/vem1.mtx --tol 1e-8
python run_assignment.py --method conjugate-gradient
```

```
# test automatici
python test_assignment.py

# stima dei numeri di condizionamento (per i commenti)
python analyze_cond.py
```

run_assignment.py rigenera results/results.csv e i grafici in results/plots/.

8 Conclusioni

La libreria implementa i quattro metodi richiesti con un'architettura coesa (classe base con lo scheletro comune + una classe per metodo con la sola regola di aggiornamento), rispettando il vincolo di usare la libreria di base solo per strutture dati e operazioni elementari. La validazione conferma puntualmente la teoria: Gauss–Seidel $\approx 2\times$ più rapido di Jacobi in iterazioni; velocità dei metodi stazionari legata alla dominanza diagonale e quella dei metodi del gradiente al condizionamento; netta superiorità del Gradiente coniugato, sempre convergente in poche decine di iterazioni e con gli errori più piccoli.